

Non-Recursive / Table Driven

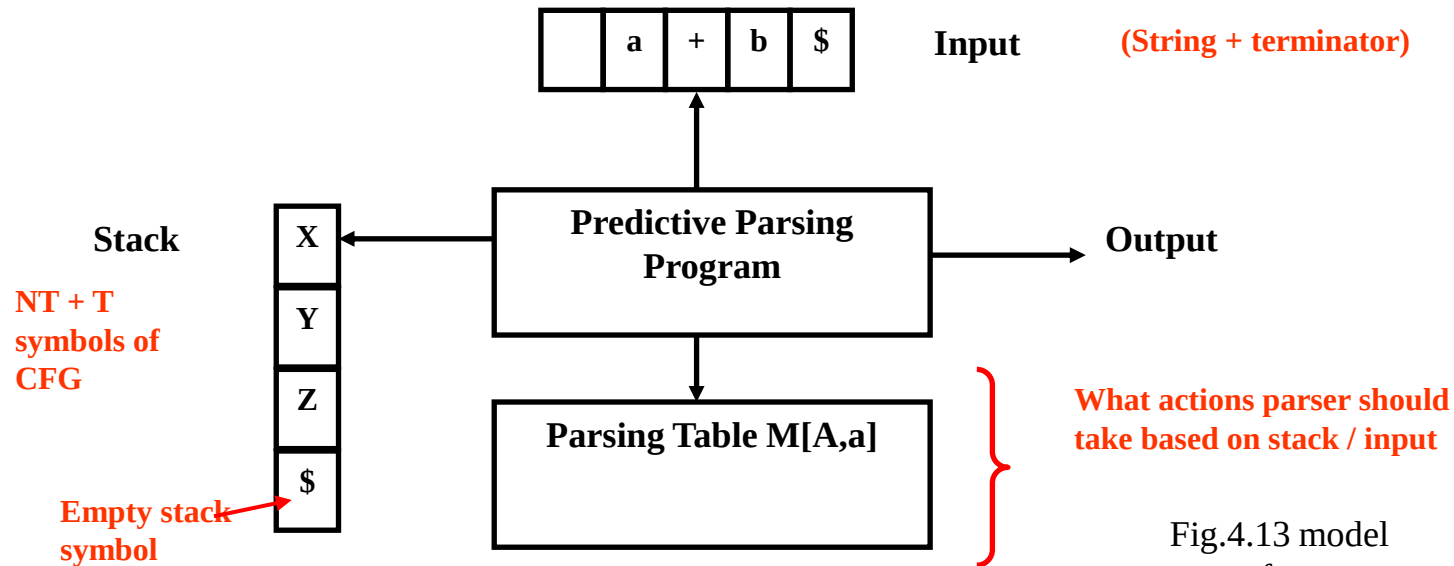


Fig.4.13 model of

General parser behavior: X : top of stack a : current input a non-recursive predictive parser (P.186)

1. When $X=a = \$$ halt, accept, success
2. When $X=a \neq \$$, POP X off stack, advance input, go to 1.
3. When X is a non-terminal, examine $M[X,a]$
 - if it is an error $\rightarrow$ call recovery routine
 - if $M[X,a] = \{X \rightarrow UVW\}$, POP X , PUSH W,V,U
 - DO NOT expend any input

Notice the pushing order

Constructing LL(1) Parsing Table

Algorithm 4.4:

1. Repeat Steps 2 & 3 for each rule $A \rightarrow \alpha$
2. Terminal a in $\text{FIRST}(\alpha)$? Add $A \rightarrow \alpha$ to $M[A, a]$
- 3.1 ϵ in $\text{FIRST}(\alpha)$? Add $A \rightarrow \alpha$ to $M[A, b]$ for all terminals b in $\text{FOLLOW}(A)$.
- 3.2 ϵ in $\text{FIRST}(\alpha)$ and $\$$ in $\text{FOLLOW}(A)$? Add $A \rightarrow \alpha$ to $M[A, \$]$
4. All undefined entries are errors.

Exercise in class

◆ Grammar:

$\text{lexp} \rightarrow \text{atom} \mid \text{list}$

$\text{atom} \rightarrow \text{num} \mid \text{id}$

$\text{list} \rightarrow (\text{lexp_seq})$

$\text{lexp_seq} \rightarrow \text{lexp_seq lexp} \mid \text{lexp}$

Please calculate $\text{FIRST}()$, $\text{FOLLOW}()$,then construct parsing table and see whether the grammar is $\text{LL}(1)$.

Step1: 消除左递归



$\text{lexp} \rightarrow \text{atom} \mid \text{list}$

$\text{atom} \rightarrow \text{numr} \mid \text{id}$

$\text{list} \rightarrow (\text{lexp-seq})$

$\text{lexp-seq} \rightarrow \text{lexp} \text{lexp-seq}'$

$\text{lexp-seq}' \rightarrow \text{lexp} \text{lexp-seq}' \mid \varepsilon$

Step2: FIRST & Follow

$$\text{lexp} \rightarrow \text{atom} \mid \text{list}$$
$$\text{atom} \rightarrow \text{num} \mid \text{id}$$
$$\text{list} \rightarrow (\text{lexp-seq})$$
$$\text{lexp-seq} \rightarrow \text{lexp lexp-seq}'$$
$$\text{lexp-seq}' \rightarrow \text{lexp lexp-seq}' \mid \varepsilon$$

$$\text{FIRST}(\text{lexp}) = \{ (, \text{num}, \text{id} \}$$
$$\text{FIRST}(\text{atom}) = \{ \text{num}, \text{id} \}$$
$$\text{FIRST}(\text{list}) = \{ (\}$$
$$\text{FIRST}(\text{lexp-seq}) = \{ (, \text{num}, \text{id} \}$$
$$\text{FIRST}(\text{lexpseq}') = \{ (, \text{num}, \text{id}, \varepsilon \}$$
$$\text{FOLLOW}(\text{lexp-seq}') = \{) \}$$

- 1 $\text{lexp} \rightarrow \text{atom}$
- 2 $\text{lexp} \rightarrow \text{list}$
- 3 $\text{atom} \rightarrow \text{num}$
- 4 $\text{atom} \rightarrow \text{id}$

- 5 $\text{list} \rightarrow (\text{lexp_seq})$
- 6 $\text{lexp_seq} \rightarrow \text{lexp lexp_seq}'$
- 7 $\text{lexp_seq}' \rightarrow \text{lexp lexp_seq}'$
- 8 $\text{lexp_seq}' \rightarrow \varepsilon$

	num	id	(	)	\$
lexp	1	1	2		
atom	3	4			
list			5		
lexp_seq	6	6	6		
lexp_seq'	7	7	7	8	

因为表项没有冲突，所以该文法是LL(1)文法。

Exercise- review LL(1)

产生式

1. $S \rightarrow AaAb$

2. $S \rightarrow BbBa$

3. $A \rightarrow \epsilon$

4. $B \rightarrow \epsilon$

LL(1)分析表

	FIRST	FOLLOW
S	a, b	\$
A	ϵ	a, b
B	ϵ	a, b

	a	b	\$
S	1	2	
A	3	3	
B	4	4	

因为表项没有冲突，所以该文法是LL(1)文法。

Exercise-Review LL(1)

产生式

1. $S \rightarrow L=R$

2. $S \rightarrow R$

3. $L \rightarrow *R$

4. $L \rightarrow id$

5. $R \rightarrow L$

LL(1)分析表

	FIRST	FOLLOW
S	*, id	\$
L	*, id	\$, =
R	*, id	\$, =

	*	id	\$
S	1, 2	1, 2	
L	3	4	
R	5	5	

因为表项有冲突，所以该文法不是LL(1)文法。

A Grammar which is not LL(1) (cont.)

- ◆ A left recursive grammar cannot be a LL(1) grammar.
 - $A \rightarrow A\alpha \mid \beta$
 - ✓ any terminal that appears in $\text{FIRST}(\beta)$ also appears $\text{FIRST}(A\alpha)$ because $A\alpha \Rightarrow \beta\alpha$.
 - ✓ If β is ε , any terminal that appears in $\text{FIRST}(\alpha)$ also appears in $\text{FIRST}(A\alpha)$ and $\text{FOLLOW}(A)$.
- ◆ A grammar is not left factored, it cannot be a LL(1) grammar
 - $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$
 - ✓ any terminal that appears in $\text{FIRST}(\alpha\beta_1)$ also appears in $\text{FIRST}(\alpha\beta_2)$.
- ◆ An ambiguous grammar cannot be a LL(1) grammar.
- ◆ What do we have to do it if the resulting parsing table contains multiply defined entries?
 - If we didn't eliminate left recursion, **eliminate the left recursion** in the grammar.
 - If the grammar is not left factored, we have to **left factor** the grammar.
 - If its (new grammar's) parsing table still contains multiply defined entries, that grammar is ambiguous or it is inherently not a LL(1) grammar.

Error Recovery in Predictive Parsing

- ◆ An error may occur in the predictive parsing (LL(1) parsing)
 - if the terminal symbol on the top of stack does not match with the current input symbol.
 - if the top of stack is a non-terminal A , the current input symbol is a , and the parsing table entry $M[A,a]$ is empty.
- ◆ What should the parser do in an error case?
 - The parser should be able to give an error message (as much as possible meaningful error message).
 - It should be recover from that error case, and it should be able to continue the parsing with the rest of the input.

Error Recovery Techniques

◆ Panic-Mode Error Recovery

- Skipping the input symbols until a synchronizing token is found.

◆ Phrase-Level Error Recovery

- Each empty entry in the parsing table is filled with a pointer to a specific error routine to take care that error case.

◆ Error-Productions (used in GCC etc.)

- If we have a good idea of the common errors that might be encountered, we can augment the grammar with productions that generate erroneous constructs.
- When an error production is used by the parser, we can generate appropriate error diagnostics.
- Since it is almost impossible to know all the errors that can be made by the programmers, this method is not practical.

◆ Global-Correction

- Ideally, we would like a compiler to make as few change as possible in processing incorrect inputs.
- We have to globally analyze the input to find the error.
- This is an expensive method, and it is not in practice.

Panic-Mode Error Recovery in LL(1) Parsing



- ◆ In panic-mode error recovery, we skip all the input symbols until a synchronizing token is found.
- ◆ What is the synchronizing token?
 - All the terminal-symbols in the follow set of a non-terminal can be used as a synchronizing token set for that non-terminal.
- ◆ So, a simple panic-mode error recovery for the LL(1) parsing:
 - All the empty entries are marked as *synch* to indicate that the parser will skip all the input symbols until a symbol in the follow set of the non-terminal *A* which on the top of the stack. Then the parser will pop that non-terminal *A* from the stack. The parsing continues from that state.
 - To handle unmatched terminal symbols, the parser pops that unmatched terminal symbol from the stack and it issues an error message saying that that unmatched terminal is inserted.

Panic-Mode Error Recovery - Example

$S \rightarrow AbS \mid e \mid \varepsilon$

$A \rightarrow a \mid cAd$

$FOLLOW(S) = \{\$ \}$

$FOLLOW(A) = \{b, d\}$

	a	b	c	d	e	\$
S	$S \rightarrow AbS$	<i>sync</i>	$S \rightarrow AbS$	<i>sync</i>	$S \rightarrow e$	$S \rightarrow \varepsilon$
A	$A \rightarrow a$	<i>sync</i>	$A \rightarrow cAd$	<i>sync</i>	<i>sync</i>	<i>sync</i>

<u>stack</u>	<u>input</u>	<u>output</u>
\$S	aab\$	$S \rightarrow AbS$
\$SbA	aab\$	$A \rightarrow a$
\$Sba	aab\$	
\$Sb	ab\$	Error: missing b, inserted
\$S	ab\$	$S \rightarrow AbS$
\$SbA	ab\$	$A \rightarrow a$
\$Sba	ab\$	
\$Sb	b\$	
\$S	\$	$S \rightarrow \varepsilon$
\$	\$	accept

<u>stack</u>	<u>input</u>	<u>output</u>
\$S	ceadb\$	$S \rightarrow AbS$
\$SbA	ceadb\$	$A \rightarrow cAd$
\$SbdAc	ceadb\$	
\$SbdA	eadb\$	Error:unexpected e
\$SbdA	adb\$	$A \rightarrow a$
\$Sbd	db\$	
\$Sb	b\$	
\$S	\$	$S \rightarrow \varepsilon$
\$	\$	accept

Phrase-Level Error Recovery

- ◆ Each empty entry in the parsing table is filled with a pointer to a special error routine which will take care that error case.
- ◆ These error routines may:
 - change, insert, or delete input symbols.
 - issue appropriate error messages
 - pop items from the stack.
- ◆ We should be careful when we design these error routines, because we may put the parser into an infinite loop.